production server. These situations are known as *environments*. When the app knows the environment in which it's running, it can load a different configuration and vary its behavior accordingly.

Before we get to that topic, let's cover the basics. What is configuration, why do we need it, and how does ASP.NET Core handle these requirements?

10.1 Introducing the ASP.NET Core configuration model

In this section I provide a brief description of configuration and what you can use it for in ASP.NET Core applications. *Configuration* is the set of external parameters provided to an application that controls the application's behavior in some way. It typically consists of a mixture of settings and secrets that the application loads at runtime.

DEFINITION A *setting* is any value that changes the behavior of your application. A *secret* is a special type of setting that contains sensitive data, such as a password, an API key for a third-party service, or a connection string.

The obvious things to consider before we get started are why we need app configuration and what sort of things we need to configure. Normally, you move anything that you can consider to be a setting or a secret out of your application code. That way, you can change these values at deploy time easily without having to recompile your application.

You might have an application that shows the locations of your bricks-and-mortar stores. You could have a setting for the connection string to the database in which you store the details on the stores, but also settings such as the default location to display on a map, the default zoom level to use, and the API key for accessing the Google Maps API (figure 10.1). Storing these settings and secrets outside your